

Building the Injection Layer: An Engineering Log

Companion to *Benchmarking AI Agents for Kaggle Competitions*

All printed numbers are real measurements as of 2026-07-30.

Wei-Hao Huang

Advisor: Tso-Jung Yen

Draft compiled July 30, 2026

Why this document exists

The main report answers what we learned about agents. This one answers what went wrong while building the machinery that produced those answers, and it is separate for a reason: the two questions have different readers and different standards of interest. A result belongs in the report if it changes what someone should believe about agent benchmarking; a defect belongs here if someone rebuilding this system would hit it too.

Every defect below shares one shape, which is the only genuinely transferable thing in these pages. **A produced artifact with no consumer is indistinguishable from a working one — in every log, every score table, and every coverage metric.** Nothing errors. The number that should have moved simply does not, and no mechanism notices. Five of the nine defects found on 2026-07-30 were of exactly this kind, and three of those were found only because fixing the previous one prompted a check of the next layer down. That progression, more than any individual bug, is the argument for tracing consumers rather than counting declarations.

The material here was originally interleaved with the results in the main report. It was moved out because a reader assessing the benchmark’s conclusions should not have to read an engineering diary to reach them — and because the diary is more useful when it is allowed to be a diary, ordered by failure class rather than by narrative convenience.

1 Judgment-Layer Coverage: Per-Regime Detail

Supporting detail for the coverage finding in Section ???. Across the 20 dossiers the judgment layer proposes 241 ideas. Most are free text, so the auditor classifies them into operator families by keyword; 39 are too vague to place at all and are counted against coverage rather than excused.

Table 1: Share of the judgment layer’s own proposals whose operator family exists, by vocabulary. “Expressible” is not “applied” — Table 2 splits these 176 by what consumes them.

Vocabulary	Expressible / 241	Share	Largest remaining gap
data joins, split_policy, postprocess	45	19%	blend (55), encoding (54), objective (22)
+ objective, blend_member	122	51%	encoding (54), feature-eng (10)
+ encoding (6 schemes)	176	73%	feature-eng (10), transform (8), structure (7)

Counting vocabulary is not the same as counting execution. Splitting the same 241 proposals by what actually consumes them separates three regimes that a single percentage conceals: those the dispatcher writes itself, those emitted as node configurations for the search driver, and those handed to the evaluator

— which the dispatcher, running on dataframes with no handle on the evaluator, books unconditionally and cannot confirm.

Table 2: The same 241 proposals, split by what consumes them. Measured by `external_data/coverage_audit.py`; percentages are of all 241.

Regime	Before	After	Who executes it
applied by the dispatcher seeded into the search advisory	12 (5%) 77 (32%) 33 (14%)	66 (27%) 77 (32%) 33 (14%)	this module writes the columns <i>nobody, before 07-30</i> ; now the arm driver the evaluator — assumed, not checked
no operator exists too vague to classify	26 (11%) 39 (16%)	26 (11%) 39 (16%)	— —

The middle regime had no reader at all until 2026-07-30: `objective`, `blend_member` and `encoding` were written into a key no search driver read, while the ledger recorded them realized. Two fixes moved the applied share from 5% to 27%: a translator (`tree_search/seed_from_ledger.py`) that turns those emissions into real seed nodes and writes back which node ids carried which operator, and moving the three target-free encoding schemes (`count`, `ordinal`, `crosses`) into the data layer where no fold boundary needs to protect them. What still cannot be executed is now stated rather than booked: `target` encoding needs per-fold computation inside the evaluator, and computing it anywhere else is the `s4e1` leakage path; `onehot_sparse` needs a linear model family the evaluators do not have.

For the advisory regime, `external_data/verify_advisory.py` checks statically whether an evaluator implements the exact parameters a dossier requested. Of the 5 advisory requests the typed dossiers make, 4 verify and 1 — `s5e10`’s `postprocess` — has no mechanism in its evaluator at all. `split_policy` is reported as structural rather than verified: folds are frozen at evaluator import time and asserted against the pipeline’s own splitter, which a static check cannot confirm.

Audit scripts: `external_data/coverage_audit.py` (regime split), `external_data/validate_dossier.py` (emission conformance), `external_data/verify_advisory.py` (advisory verification).

2 Reconstruction of the Voided Ventilator Attempt

The driver log and the lane’s own artifacts reconstruct the failure precisely — and it is an *execution-discipline* failure, not a time-budget one. The harness granted the full 4h budget; the session lasted **5 minutes** and exited cleanly. The frozen vote rule selected the highest-voted kernel (a literal *starter* tutorial, `theoviel/deep-learning-starter-simple-lstm`, 1,166 votes), the agent ported it quickly, then launched the 200×5-epoch training *in the background* — violating its explicit foreground-only instruction — logged “training running... waiting on monitor events,” stopped calling tools, and thereby ended its own session; the orphaned training process died after **2 of 1,000** fold-epochs (37.6s of training; fold-0 validation MAE 1.28 → private 8.56), and the salvage submission shipped with no sanity gate. Under the fairness protocol the result stands: the environment delivered its budget, and the failure belongs to the frozen method’s execution. The re-run, with the prompt hardened against the backgrounding pattern, used 109 minutes and scored 0.40093 — so the method itself is competitive on this task, and the 8.56 measured only the lapse. The episode extends the “self-assessment cannot veto output” family (Section ??) with a new member: *process discipline* (foreground execution, session-lifetime awareness) is itself part of agent capability, and one-shot reproduction is all-or-nothing when it lapses, while iterative search always holds a submittable solution.

3 Silent Failures Encountered and Fixed

During the study we found five classes of failure that raise no errors; all were fixed and mechanized before the reported runs (Table 3).

Table 3: Five silent failures of the experimental design, their detection, and fixes.

#	Failure	Impact	Detection	Fix
1	Harness copied my-agent’s entire working directory to AIDE, exposing processed features and tuned hyperparameters	4 AIDE results voided	Inspecting which files the champion code actually opened	Manifest-validated isolated roots; 4 competitions re-run
2	Runs under resource contention (one lane at load 17, another with the machine to itself)	Unequal conditions	Comparing lane execution windows	<code>lane_lock.sh</code> mutex
3	Watchdog judged health by “machine busy overall”; a dead lane went unnoticed all night	One night of capacity lost	Per-driver check (no completion marker, no process)	Watchdog extended with per-driver checks
4	Aggregation took “best score across all experiments,” counting leakage experiments tagged as diagnostic	3 competitions’ numbers wrong	Scanning logs for exclusion tags	Fixed and disclosed
5	Headless-agent instructions enumerated six stages but omitted tree search; the agent followed the prompt, not the skill	5 competitions ran a truncated method	Checking for <code>experiments_tree*.json</code>	Prompt changed to “follow the skill; this prompt is not a pipeline definition”

Measured contrast for failure 5 (afsis-soil-properties): the truncated run produced 0 tree-search nodes, CV MCRMSE 0.44817, in 32 minutes; the fixed run produced 80 nodes, CV MCRMSE 0.444076, in 17 minutes. After the fix, the score improved *and* the run got faster — consistent with the skill’s internal evidence that tree search wins 9 / ties 1 / loses 0.

4 Isolation: The Mechanisms, and Every Time One Failed

The main report states four isolation conditions and what enforces each, because the three-way comparison is meaningless without them. This section is the other half: how each is built, and the record of every occasion one of them broke. That record is the more useful half — each failure was silent, and each was found by a different accident.

4.1 Data isolation

`build_isolated_roots.py` constructs one root per competition per side, linking only files the Kaggle API’s own manifest lists as official — including members extracted from official archives, which is the case a naive filename check gets wrong. Officialness is judged by the manifest rather than by convention, because the failure it exists to prevent looked exactly like a normal working directory.

Failure 1. The harness copied my-agent’s entire working directory to AIDE, exposing processed features and tuned hyperparameters. Four AIDE results were voided and re-run. Detection was not automatic: it came from inspecting which files the champion code actually opened. Nothing in the logs distinguished a lane reading its own features from one reading somebody else’s.

4.2 Resource isolation

`lane_lock.sh` is a mutex built on atomic directory creation, holding the owner's PID so a dead holder's lock is reclaimed rather than wedging the queue, with bounded waiting and a timeout that logs contention explicitly rather than proceeding in silence.

Failure 2. Before the mutex existed, the rule was written down but not enforced: one lane ran at load average 17 while another had the machine to itself, on the same competition. This is the defect that voided RUN1 and is the reason the mutex was written.

Failure 2b, found 2026-07-30. The mutex was a no-op for the one driver that called it from Python. It acquired inside a throwaway `bash -c` subshell, whose EXIT trap fired `lane_release` the moment the command finished — and the ownership test passed, because that subshell genuinely was the recorded owner. `lane_lock.log` shows ACQUIRED and RELEASED stamped in the same second while the training it was supposed to protect ran for another 95 minutes. No overlap resulted, purely because nothing else tried to start. Fixed by holding the lock in the long-lived parent process, in an implementation wire-compatible with the shell version so the two exclude each other rather than each honouring a private lock — verified by having a shell acquirer refused while Python holds it.

4.3 Instruction isolation

The rule is that a prompt may carry environment facts and metric thresholds only: no modelling recipes, no other side's scores, no workaround discovered by another side. This one is enforced by protocol and an incident log rather than by code, which is a real weakness and is disclosed as one — there is no mechanical check that a prompt is free of leaked strategy.

Failure 5. The headless driver's prompt enumerated six pipeline stages but omitted tree search, and the agent followed the prompt rather than the skill. Five competitions ran a truncated method. The measured cost on afsis-soil-properties: the truncated run produced 0 tree-search nodes and CV MCRMSE 0.44817 in 32 minutes; the corrected run produced 80 nodes and 0.444076 in 17 minutes — better *and* faster. The fix was to make the prompt state that it is not the pipeline definition: *follow the skill as written*.

Failure 5b, found 2026-07-30. A driver launched AIDE without pinning `exec.timeout=1800`, so it silently fell back to its own 3600s default — twice the per-step execution budget every comparable run had used. Caught by a condition check in the successor driver, which compares each run's recorded `timeout` and `steps` against the pinned benchmark values before counting the run as conforming. The affected runs were re-done. This is why AIDE lanes must be launched through the existing drivers rather than a hand-assembled command line.

4.4 Scoring integrity

Failure 3. The watchdog judged health by whether the machine was busy overall, so a dead lane went unnoticed for a night while other work kept the load high. Fixed with per-driver checks: no completion marker and no live process means dead, regardless of what else is running.

Failure 4. Aggregation took the best score across all experiments, which counted leakage experiments that had been deliberately tagged as diagnostic. Three competitions' numbers were wrong. Found by scanning logs for the exclusion tags that were already there and already being ignored.

On the shape of these. Failures 1, 3 and 4 all had the same structure as the injection defects in the previous section: the artifact existed, the log looked normal, and the number was wrong for a reason no mechanism was watching for. In each case the evidence needed to catch it was already being written down — which files were opened, which processes were alive, which experiments carried an exclusion tag — and nothing was reading it.

5 The Nine Defects of 2026-07-30

A single day’s audit, recorded because the pattern matters more than the individual entries. Six were surfaced by a deliberate adversarial audit of the v5 stack; three more were found while verifying the fixes for the first six — which is itself the finding.

1. **The experience library handed a rerun its own recorded answers.** The agent is instructed to consult `knowledge/experience.md` before EDA and before modelling, and retrieval matched on metric and task tags only. By July the library held the benchmark runs’ own conclusions, so re-running a competition would have returned the feature set and ensemble choice that produced its recorded score. Fixed by dropping bullets whose evidence field names the competition being solved — the citation was already in the text, which is why the fix was cheap. On aug-2022 it drops 4 of 9 matching bullets, including the line carrying that competition’s own final score.
2. **The stored prior mandated the operator form that loses.** TASK-TS-FUTURE required `ratio_target` and forbade `join_feature` alone, on the sound argument that a tree cannot extrapolate a feature past its training range. Two leaderboards say otherwise. The prior now requires racing both forms under a rule fixed in advance, and records the counter-evidence.
3. **Three operators were emitted into a key nothing read.** `objective`, `blend_member` and `encoding` were written to `plan["node_configs"]` and booked “realized”; a repository-wide search found the write, the docstring describing the contract, and no reader. Fixed with a translator that turns them into real seed nodes and writes back which node ids carried which operator.
4. **LightGBM determinism was unpinned.** Scores are stored to six decimals and the champion is a bare min, so the default multi-threaded histogram build could flip selection on cross-process variance. Pinned across all ten evaluators.
5. **External-data vintage was unpinned.** The cache keyed on indicator and year range but not vintage, so a refresh could substitute a revised series — and the World Bank rebases constant-price GDP periodically, rewriting all of history. Now a tracked lockfile with an assertion.
6. **carry_forward filled backwards in time.** Unmatched rows were filled with the panel’s single latest year regardless of direction, which is correct for a genuinely-future test row and a leak for an early training row. Now checked per row.
7. **postprocess spoke a different vocabulary than its evaluator.** The dossier asked for `round_to_int / clip_min / global_scale`; maybe `postprocess` recognised only `auto_scale / scale`. Requests were silently dropped. Found while verifying the fix for defect 3.
8. **The ledger claimed a verification that never ran.** `split_policy` and `postprocess` were booked “honored by the evaluator” unconditionally — the dispatcher runs on dataframes and cannot see the evaluator. A static checker now reports 4 of 5 such requests implemented and 1 with no mechanism at all. Found while writing up defect 7.
9. **A lazy-evaluation refactor silently disabled ratio_target.** Making panel-key resolution lazy (the fix for a different problem) left one call referencing the removed eager variable, raising inside the operator’s own try block. Invisible on arms built before the change; the first arm built after it would

have shipped a “ratio” arm with no ratio in it. **The coverage ledger caught this one** — it appeared as an explicit unrealized entry with the exception text rather than as a plausible-looking arm. An AST-based guard now asserts no bare panel-key name survives in that function.

What the sequence shows. Defects 7, 8 and 9 were each found while verifying the fix for the one before it. None would have been caught by running the system and looking at scores, because in every case the scores were plausible. They were caught by asking, of each declared capability, *which line of code consumes this?* — and following the answer until it either reached an execution site or ran out. That question is cheap, mechanical, and was not part of the process until this day.

One defect was caught by the architecture rather than by a person. Defect 9 surfaced as a ledger entry because the coverage ledger was built, after defect 3, to record unrealized operators with their reasons. That is the only instance here of the system catching its own failure, and it is the argument for the ledger existing: the eight others required someone to go looking.